

Team sprint: October 3-5, 2026

Start with the [full agent sprint plan](#), or read its [portable PDF](#). The [aggregate audit](#) preserves the measured findings, source revisions and SQL.

Status: proposed execution plan. Work packets are planned, not completed. **Scope:** ETL, backend and frontend. ML implementation and benchmarking belong to the ML team; backend integration with its delivered interface is included. **Calendar:** October 3-5, 2026, America/Guatemala. Dates are release checkpoints; dependencies and acceptance evidence control progression. October 5 is the requested planning horizon, not a verified event submission deadline.

GitHub review update

Read the [branch/PR review](#) and its [machine-readable evidence](#). Verified account: yehosuah. There are no open PRs; ETL PR #2 and backend PR #1 are already merged. Unmerged ETL and backend branches contain useful work to reuse, while the frontend remains a starter.

The newest backend branch passed all 215 tests with disposable PostgreSQL. Three additional probes found an accepted-case recovery gap, terminal cases ahead of active queue work and contradictory receipts marked verified. Treat these as integration work in P00/P04/P08/P09/P10. The ETL candidate passed 187 tests and its profiler ran on the existing accepted release.

Start by pinning and reviewing the candidate branch chain, then reuse its profiler, optional ML packaging, tool dispatch, metrics and persistent cases. The full plan remains a proposed execution plan; branch implementation alone does not pass a packet's end-to-end gate.

What we are building

One polished card-support experience in Spanish and Portuguese, with three demonstrable routes:

1. Confirm and execute a temporary card pause, then show the verified result.
2. Clarify an ambiguous block request before selecting a card or taking action.
3. Register an unrecognized-charge review request and produce a structured support case.

Every action is simulated, customer-scoped, confirmed where required and backed by an execution receipt. A registered request is not a refund, card shipment or fraud decision.

Checkpoints

Date	Required outcome
October 3	Executable contracts, reproducible synthetic ETL-to-API boot, browser session/card context and the first verified action journey.
October 4	Clarification, loss/theft handling, charge-review handoff, failure recovery, refresh/restart correctness and browser polish.
October 5	Clean setup, accessible deployment, real ML integration acceptance, frozen evidence, public submission package, slides and mandatory demonstration video.

Start here

Start **P00: contracts and pinned revisions** and **P03: data rationale and quality evidence** in parallel. After P00, complete **P01: synthetic ETL-to-API boot**. Then P02, P04 and P10 can run in parallel with distinct frontend, conversation and pipeline ownership. P05 combines the first verified browser action journey.

Use one integration owner and up to three active worker agents. Each packet has one owner and isolated working state. The integration owner integrates changes serially and owns the shared acceptance stack. The full plan lists all sixteen packets, dependencies, acceptance details and reusable agent kickoff prompts.

Repository map

Repository	Responsibility
FactoredAI_base	ETL, data contracts, fixtures, quality/freshness evidence and integrated runtime packaging.
FactoredAI_BCK	Sessions, conversations, permissions, confirmations, tools, receipts, handoff and ML adapter.
FactoredAI_FRT	React customer experience, ES/PT copy, typed API client and browser verification.

Clones can live anywhere. Keep the repositories and environments independent, pin their revisions and integrate through the agreed contracts.

External prerequisites

The final deployed release needs hosting access, a supported public TLS address, any required spending authorization, a compatible real ML delivery, the ML team's evaluation evidence and the submission team name. A stub-backed engineering pass does not complete the hackathon submission.

Live S3 activation remains deferred. Static historical inputs and labeled update fixtures are sufficient for this sprint's engineering demonstration.

The GitHub plan requires access to this private repository. The PDF and download bundle can be forwarded independently. Share the plan and aggregate evidence; organizer records, raw transcripts and credentials are not included.

Agent sprint: October 3-5, 2026

Status: proposed execution plan. Existing main and candidate-branch implementations are distinguished below; no work packet is accepted solely from branch presence. Dates use America/Guatemala. October 5 is the requested sprint horizon, not a verified organizer submission deadline.

Outcome and scope

A judge can open the deployed application, enter an isolated synthetic test session, and complete a card-support conversation in Spanish or Portuguese. The application asks for missing information, obtains explicit confirmation for mutations, executes an authorized simulated tool, verifies its result, and presents a useful support case when escalation is required. A fresh clone can reproduce the demo and its engineering acceptance evidence.

This sprint covers **ETL, backend, and frontend**. Model selection, training, prompts, intent classification, retrieval algorithms, annotation, translation certification of ML datasets, model tuning, and model benchmarking are outside this sprint. Backend integration with the ML team's delivered interface is in scope. Engineering tests of that interface and its failures are in scope.

All work gates below are agent-verifiable. They do not depend on meetings, human estimates, story points, or scheduled review ceremonies. Hosting access, permitted data use, spending authorization, the team name, and a real compatible ML delivery are external prerequisites; agents cannot manufacture them. The deployment architecture is provider-neutral, as requested.

What exists, and what is missing

The audit used local source and the original problem statement and kickoff PDFs, not the presence of a README alone. The [aggregate audit record](#) preserves source revisions, table reconciliation, the SQL behind the attribution findings and check results without individual records.

Area	Evidence observed on October 3	Gap to close
ETL	Nine-table extraction, typed transformation, quarantine, immutable Parquet, transactional PostgreSQL publication, scheduling and recovery are implemented. Accepted release has 6,020,768 rows; 93,261 of 6,114,029 inputs are quarantined.	Reproducible public synthetic boot, usable quality/freshness evidence, card-workflow rationale, and CI that does not depend on private local files.
Backend	FastAPI supports opaque sessions, customer-scoped reads, masked numbers, persistent simulated card actions, eligibility checks, transactional idempotency and action auditing.	Conversation lifecycle, ML adapter, server-side confirmation, complete case handoff, conversation traces, stable domain errors and public-demo isolation.
Frontend	React/TypeScript/Vite starter has ES/PT copy and a language selector.	Authentication, card context, chat, clarification, confirmation, verified receipts, handoff, failure states and API integration.
ML dependency	ETL delivers an exploration pack: 16 policies, 72 intent cases, 20 retrieval cases and 16 handoff cases. Labels are drafts; splits are unset. No ML runtime is integrated in these checkouts.	ML team supplies a compatible runtime and its own baseline/held-out evidence. Do not convert this into ML work in this sprint.
Operation	October 1 evidence documents real ETL/BCK publication, rollback, customer isolation, concurrency and restart checks.	Re-run acceptance on the final integrated build and deploy an externally reachable demo. At audit time Compose had no running services and port 8000 did not respond.

Area	Evidence observed on October 3	Gap to close
Submission	Requirements exist in the local briefs.	Public submission repository, deployment link, 4-6 slides, mandatory short demonstration video and a coherent evidence index.

Checks run during this audit:

- ETL make check passed; make test passed 207 collected cases. Five byte-identical * 2.py test copies account for 100 cases; the canonical suite collects **107**, not 207 independent tests.
- Backend make check passed, including **22** tests, in its own environment.
- Frontend npm ci --offline --no-audit --no-fund and npm run check passed. This establishes a buildable starter, not a working service flow.
- Thirty tracked runtime/test/lock files compared between the workspace ETL and FactoredAI_base are byte-identical. They are separate checkouts of the same ETL, not two pipeline modules.
- Real PostgreSQL/restart results in the October 1 verification record are historical evidence; this audit did not restart services or claim a new live end-to-end pass.

Specific findings that change the sprint:

1. **The current demo cannot exercise a complete unknown-charge flow.** Default fixtures create cards and states, but no movements. Movement reads and charge registration use bank.transactions. The existing verification script exercises disputes with an organizer-backed test account.
2. **A fresh clone lacks local prerequisites.** Organizer data, the accepted raw release and ML draft files are ignored. The transform library allows no ML pack, but the CLI defaults to a private local ML-pack path. Public-demo boot must explicitly handle this without changing ML content or readiness claims.
3. **Handoff is incomplete.** /me/handoff returns the customer's latest committed action results. It does not capture a conversation's request, unresolved questions, failed attempts or reason for escalation, and it is not a support queue.
4. **Data attribution has a serious limitation.** The accepted interactions contain 536,046 comma-separated product-mention tokens. Exact joins find 3,475 products, including 1,197 card products, but zero matched mentions also have the interaction customer's ownership. This is a failure of that attribution method, not evidence of zero card demand. Do not use it to authorize access or count card-support calls.
5. **Quarantine affects analytics.** Complaints lose 44,878 of 67,095 input rows (**66.9%**). Transcripts lose 27,469 of 171,321 (**16.0%**). Almost all customer branch references remain explicitly unresolved. Retained-only complaint or branch statistics can be misleading.
6. **Broad contact reasons are insufficient routing labels.** The 670,427 accepted interactions have six broad categories; Producto and Transaccional do not mean card support. There are 140,040 accepted card products across 91,084 customers, but product prevalence does not establish support-call incidence.
7. **Documentation is stale in places.** The brief and evaluation README still describe some implemented work as pending. Agents must use current code, contracts and verification records as their implementation baseline.

Relevant sources:

- [Original problem statement](#), pp. 2-6: focused flow, ES/PT, normal/ambiguous/human-required routes, verified tools, permissions, learned-component comparison, held-out failures and operational evidence.
- [Original kickoff](#), pp. 10-15, 18 and 20: working system and submission artifacts. No numerical judging weights are assumed.
- [Accepted ETL/BCK contract](#), [October 1 verification](#), [runbook](#), [ML review state](#).

- Backend source: routes.py, store.py, security.py, its own README and ADR 0001. Frontend source: App.tsx, i18n.ts, its own README and docs/reto.md.

Remote branch review: reuse before implementation

The [GitHub review](#), checked through yehosuah, supersedes any inference that the initial local checkouts represent all work in progress. There are zero open PRs in ETL/BCK/FRT. ETL PR #2 and BCK PR #1 are already merged. Candidate ETL de0d3ea adds a working aggregate profiler, optional ML packaging and auxiliary-file hardening. Candidate BCK fd40d2c includes the entire observability -> backend-tools -> human-handoff-routing chain. FRT remains 95a1a43 with no additional branch.

ETL checks and 187 tests passed; its profiler ran successfully against the accepted release. The latest BCK passed 215 tests with disposable PostgreSQL. Additional probes reproduced **R1** accepted-case recovery after account disablement, **R2** terminal-case queue ordering and **R3** contradictory action receipts marked verified. Fix these before final integration. Handoff grants/preflight are separate deployment prerequisites; the current integrated boot has not been updated to apply them.

Revise dispatch without adding a second implementation of existing work:

- **P00:** pin accepted main and candidate revisions separately, carry R1-R3 into owned BCK work, and specify handoff grants/readiness and the confirmation/conversation contracts.
- **P01:** reuse optional ML packaging; still implement complete synthetic ETL boot, movements, source-aware provenance, isolated visitor state and support-account provisioning.
- **P03:** reuse the profiler and its accepted-release output; finish workflow rationale, quality/attribution limits and freshness evidence. It measures 1,547,408 ownership-consistent card transaction rows and 77,712 recorded declines, which do not establish support demand or financial benefit.
- **P04/P05/P09:** reuse ToolDispatcher, correct receipt-state validation and add persistent conversations, delivered ML adapter, server-owned confirmation and unknown-outcome reconciliation.
- **P08/P10:** extend existing cases with conversation/attempt references, authorized recovery, active/manual queues, support UI and release/restart continuity.
- **P11/P13:** reuse process-local metrics but add end-to-end timing, attempted-failure evidence and CI that cannot silently skip database acceptance.

The profiler also shows 42 exact customer texts and 546 exact full texts in 143,852 accepted transcript rows. Keep this aggregate quality limitation visible; do not label row count as independent conversational evidence. No model implementation or evaluation work is added to this sprint.

The full review contains immutable code links, inventory, severity, reproductions, validation conditions and merge recommendations. The original local audit remains a historical snapshot; the new [remote review JSON](#) records the branch evidence independently. No packet is marked complete yet.

Product strategy

The flagship demonstration is one coherent card-support journey with three routes:

1. **Normal:** identify an owned active card, confirm a temporary pause, execute once, show the verified PAUSED state, and optionally reactivate it.
2. **Ambiguous:** receive “block my card” / “bloqueie meu cartão,” clarify temporary pause versus loss/theft and select the correct owned card before showing any action confirmation.
3. **Human-required:** select an owned historical or team-synthetic transaction, register an unrecognized-charge request, and create a structured support case. State that review was requested; do not claim a refund or fraud adjudication.

Also prove loss/theft blocking cannot be automatically reversed. Keep activation and replacement available only if they meet the same acceptance standard; they are secondary polish rather than prerequisites for the

three flagship routes.

Make elegance visible through one consistent interaction: card context, a conversation, and concise result/case panels. Clearly separate “needs your confirmation,” “result verified,” “request registered,” and “transferred to the support queue.” Show source dates and simulation provenance near the relevant values. Demonstrate explanations from policy versions, source references and execution receipts.

The strongest differentiator available in this codebase is trustworthy action execution backed by a reproducible data release. Broadening to lending, payments or other banking workflows does not solve the current gaps and has no automatic judging bonus.

Repository ownership and concurrency

Boundary	Working location	Ownership
ETL	FactoredAI_base	Pipeline, fixtures, release contract, quality/freshness aggregates, integrated runtime packaging. Use one canonical checkout per packet.
Alternate ETL checkout	Any second clone of FactoredAI_base	Same ETL project. Pin one checkout for each work packet; never apply the same change to both.
Backend	FactoredAI_BCK	Sessions, conversations, authorized tools, confirmations, verified receipts, cases and ML interface adapter. Independent Git/environment/checks.
Frontend	FactoredAI_FRT	React customer experience, translation copy, typed client, browser checks. Use the inner Git repository.
ML	External team's delivery	Runtime and ML evidence. Consume its interface; do not implement its internals.

Use an integration owner plus up to three active worker agents when this environment's four-agent limit applies. With separate chats, keep the same work-in-progress cap as an organizational rule. Role names are ownership labels, not a request to spawn chats now.

- **Integration owner:** pins revisions, maintains the contract and dependency board, selects ready packets, integrates changes serially, owns the shared acceptance stack and release evidence.
- **Data/runtime worker:** ETL boot, quality report, freshness and reproducible release packaging.
- **Service worker:** backend lifecycle, tools, confirmation, adapter and handoff.
- **Experience worker:** frontend integration and user-facing behavior; later rotates into independent browser QA.

Each packet has one owner, even when it spans repositories. A packet may request a bounded contribution from another worker, but two workers must not edit the same files or run mutating tests against the same database concurrently. Use isolated worktrees and temporary fixture databases. Only the integration owner runs release/restart checks against the shared stack.

Every handoff includes repository/branch and commit IDs, contract version, behavior demonstrated, exact commands and results, fixture provenance, remaining blockers and affected consumers. Use the yehosuah/ branch prefix. Preserve existing untracked work and duplicate files until compared; do not clean the workspace by bulk deletion.

Keep implementation tickets and decisions in each repository's configured local issue area. This document is a cross-repository proposed plan, not a publication of issues or a modification of existing completed tickets. Put shared integration dependencies in the plan's board and reference their IDs from repository-local tickets when execution begins.

Gates and calendar anchors

The dates organize release checkpoints. They are not task-duration estimates. A gate passes only on evidence; a calendar change never overrides an unmet dependency.

Anchor	Gate	Observable exit condition
October 3	G0: reproducible foundation	P00-P03 complete: pinned repositories, executable contracts, public synthetic ETL/BCK boot, real browser login/card reads, traceable data rationale.
October 3 onward	G1: first complete journey	P04-P05 complete: a stored conversation can confirm and execute a pause once and show its verified result in the browser. Stub versus real ML mode is explicit.
October 4	G2: complete behavior	P06-P09 complete: clarification, loss/theft, transaction review and structured escalation, plus timeout/replay/expiry behavior, pass in ES and PT.
October 4 onward	G3: operation and polish	P10-P12 complete: corrected-release/restart acceptance, measured runtime evidence, polished browser experience and evidence ready for P13 release packaging.
October 5	G4: judges' release	P13-P15 complete: reachable TLS demo, real ML integration acceptance, final public package, slides and working video validated against the frozen revisions.

Foundation and independent analysis run concurrently. P00 includes frontend consumer examples against the executable contract; the complete browser packet starts after synthetic boot passes. Documentation and submission structure can be prepared early; claimed results, screenshots and video must come from the accepted build.

Dependencies are listed in the work-packet table below. The shared Markdown guide also includes the dependency diagram.

Agent work packets

P00-P11 and P13-P15 are required. P12's accessibility and coherent failure presentation are required; decorative additions are optional. P07's loss/theft behavior is required because the existing simulator supports it and confusing it with temporary pause is material.

ID	Owner; repositories	Blocked by	Demonstrable result
P00	Integration; BCK + contract mirror in FRT/ETL	Nothing	Pinned versions and an executable consumer contract exercise success, clarification, confirmation, escalation and invalid ML responses with a labeled stub. Existing HTTP tools remain compatible.
P01	Data/runtime; ETL + BCK	P00	A new empty demo database ingests a versioned team-generated nine-table fixture through Extract/Transform/Load and serves owned cards and movements through authenticated BCK endpoints. No organizer files, AWS access or private ML pack are needed.
P02	Experience; FRT + BCK	P00, P01	Browser enters a bounded, isolated trusted test session, shows its cards/movements and source date, and handles missing release, expiry and logout. Two visitors cannot modify each other's demo state.
P03	Data/runtime; ETL	Nothing	Reproducible aggregate report explains workflow choice, population/coverage, quarantine and attribution limitations, using a pinned organizer release locally and no individual records in the public output.

ID	Owner; repositories	Blocked by	Demonstrable result
P04	Service; BCK	P00, P01	Authenticated conversation creation/turns persist across reconnect, enforce ownership, call an injected ML adapter, validate structured outputs and correlate turn/tool/evidence IDs. A stub-backed HTTP demonstration verifies the interface.
P05	Service/experience packet owner; BCK + FRT	P02, P04	User requests a pause, confirms the exact card/action, and sees a verified receipt and refreshed state. Duplicate submission causes one effect. Cancel, changed selection and stale confirmation do not execute.
P06	Experience packet owner; FRT + BCK	P05	Ambiguous requests stop before action; card selection and pause/loss clarification continue the same conversation. ES/PT language changes preserve context and do not change an already confirmed action's meaning.
P07	Service packet owner; BCK + FRT	P05	Explicit loss/theft confirmation blocks an owned card and displays a verified BLOCKED result; attempted automatic reactivation is refused with a useful next step. Multiple requested actions cannot bypass priority or eligibility.
P08	Service/experience packet owner; BCK + FRT	P05, P01	User selects a fixture movement, registers human review, and opens a conversation-scoped support case with request, verified facts, actions/attempts, evidence and unresolved questions. Restricted demo support view can inspect it.
P09	Service; BCK + FRT	P04; P05 for action checks	ML/tool timeout, malformed provider response, lost HTTP response, expiry and retry are reproducible. UI distinguishes failure from unknown outcome and obtains the committed receipt before asserting success. No automatic mutation from fallback.
P10	Data/runtime; ETL + BCK	P01	Controlled correction and partial-load failure keep readers on a complete release; repeat and restart preserve existing simulator state and action receipts. Source cutoff stays historical while pipeline execution time changes.
P11	Independent acceptance worker; all three	P03, P06-P10	Automated browser/API/Compose engineering suite emits a versioned results bundle, failure evidence, latency distributions and capacity observations without source rows or secrets.
P12	Experience; FRT	P06-P09	Final UI makes context, confirmation, receipts and support cases legible on desktop/mobile; keyboard, focus, contrast and all ES/PT states pass browser inspection.
P13	Integration/runtime; all three	P11, P12	Empty-directory clone/build/boot works from pinned independent repos with public synthetic fixtures, CI checks and a release manifest; no private workspace paths are required.
P14	Integration; all three	P13, hosting access, compatible real ML delivery	Externally reachable TLS deployment passes the three ES/PT routes with the real delivered ML interface and isolated demo identities. Missing ML or hosting remains an explicit blocker, not a passed gate.
P15	Integration/documentation; public submission package	P03, P14, ML team's evidence, team identity	Required repository link, deployment link, 4-6 slide presentation and short demonstration video match the accepted build and accurately separate engineering evidence from ML evidence and projected business impact.

Acceptance details for the critical packets

P00 - Small interfaces, explicit authority. Use IMPROGRESS Desarrollo's module-design guidance: one injected ML interface with a real adapter and a labeled stub; one authorized workflow interface around existing tools. Freeze request/response schemas and representative fixtures before consumers diverge. Define conversation and turn IDs, ES/PT language, minimal allowed context, intent/action proposals, clarification data, evidence references, provider version, errors and deadline behavior. Include proposed customer HTTP contracts for turns, confirmation, receipts and support cases. These are new proposed contracts, not currently implemented routes. The model cannot supply the trusted principal, grants, eligibility, action success or refund/issuance authority. No direct access from the browser to model credentials or private databases.

P01 - Real data path for the public demo. Preserve all nine-table grains, original fixture CSVs, checksums and release reconciliation. Include an active card and eligible/blocked states, owned movements with transaction ID + process date, a second isolated customer, missing optional amounts and a correction fixture. Emit explicit team provenance. Backend reads from bank.products currently hardcode organizer provenance; derive the correct provenance from the release manifest for fixture releases. Add an explicit CLI/demo setting to omit the optional ML review pack while preserving existing historical-pack behavior. Do not silently fabricate a missing organizer release or fall back from organizer mode to fixture mode.

P02 - Isolated test identity. Keep identity established by the trusted backend; a typed customer ID is never authentication. Public "try demo" sessions, if enabled, are restricted to bounded team fixtures with separate customer/card state, expiry and cleanup. Organizer-account provisioning stays private/administrative. Clear conversation and card data on logout or identity change. Prefer a same-origin /api proxy; document development proxy and production routing explicitly rather than relying on absent CORS behavior. Keep tokens out of URLs, logs and public bundles.

P04/P05 - Action authority. The backend stores a pending action bound to principal, conversation, card, parameters, policy version, expiry and relevant eligibility/state. Confirmation references that pending action; it is not an arbitrary browser or ML boolean. Revalidate before execution. Reuse existing transactional ownership/idempotency rules. Keep the same idempotency identity across retry of one confirmed command and reject conflicting reuse. Verified receipts use committed tool evidence. A server-owned typed receipt, not free-form model prose, establishes account facts or action completion.

P08 - A real support case within a simulated bank. Preserve actual conversation scope rather than aggregating a customer's last 20 unrelated actions. Separate verified facts, requested actions, failed/unknown attempts, committed outcomes and unanswered questions. Include policy/source/execution references, provenance, creation time and status. Require a support role for queue access; customer sessions can see only their own case. The queue is a prototype for handoff; it does not demonstrate a real employee accepted or resolved a case. Do not require an external CRM or send messages to people.

P09 - Bounded failures. Define adapter/tool deadlines and a bounded retry policy. Retry transient reads/provider calls only within their budget. Mutating retries keep the original command identity and reconcile receipts; they never create a new business command automatically. A timeout after commit produces an unknown client outcome until reconciliation, not a false "failed" or "completed" claim. Auth expiry revokes further access; a stale confirmation requires reauthentication and revalidation. Test hostile instructions at the engineering interface: another customer's IDs, fabricated evidence, unsupported tools and unauthorized action proposals must fail regardless of model output. This is service-boundary testing, not an ML prompt-injection benchmark.

P10 - Historical truth and refresh correctness. Preserve the accepted ADRs: immutable releases, transactional pointer, separate simulator state, branch-quality flags and optional money. Do not weaken ownership checks to improve acceptance counts. Show event/process/source cutoff separately from publication and scheduler-run time. A recent offline job is not fresh banking data. Live S3 remains deferred and is not required for the demo; labeled correction fixtures establish update behavior, as allowed by the problem statement.

P11 - Evidence rather than checkmarks. Cover the three routes in both languages, customer and conversation isolation, stale confirmations, idempotent races, malformed ML replies, expired sessions,

missing data, tool timeout, partial publication and restart. Verify stored conversations, cases and receipts survive restart. Compare frontend state with authoritative API/DB receipts; assert absence of forbidden effects. Capture observed workload, sample size, warm/cold conditions, concurrency and end-to-end p50/p95, plus backend/DB timings. Publish observed values rather than inventing organizer thresholds. The internal gate is zero observed unauthorized effects, false action completions or false refund/issuance claims in the declared engineering suite, with counts and denominators; passing does not establish zero general risk. Record unresolved failures. ML correctness, learned-component comparisons and scored held-out evaluation stay with the ML team.

How agents take work

1. The integration owner pins a base commit and contract version and chooses a packet whose dependencies passed. P00 and P03 can start immediately, reusing the candidate branches documented in the remote review. After P00, start P01 while P03 continues if needed. After P01, P02, P04 and P10 form a useful three-worker frontier with separate frontend, conversation and pipeline ownership.
2. A worker claims one packet and one isolated branch/worktree. Read that repository's AGENTS, README, glossary, ADRs and relevant contracts. ETL uses its own uv/Python 3.13 environment; backend uses its own uv/FastAPI environment; frontend uses its Node 24/npm lockfile.
3. Implement the smallest complete demonstrated behavior. Validate at the external interface, using existing fixture/HTTP/browser seams. Do not treat an unimplemented dependent layer as complete because a function unit test passes.
4. An independent worker verifies the handoff evidence and integration owner incorporates it serially. Resolve contract changes before letting affected consumers proceed.
5. Update packet state and release evidence. Stop/reassign a blocked packet to useful ready work; never fill its dependency with an unlabeled fake and mark the gate passed.

Suggested kickoff for a worker:

> Implement packet <ID> from this sprint plan in its listed repository or repositories. Read their instructions and accepted contracts first. Own only this packet's agreed files and use isolated fixtures/state. Do not work on ML algorithms, prompts, labels, training or model evaluation. Demonstrate the specified external behavior, run the repository's required checks and affected integration checks, and hand back commit IDs, contract version, commands/results, evidence and blockers. Do not publish data or spend money. Coordinate interface changes with the integration owner before modifying consumers.

Suggested kickoff for independent acceptance:

> Verify the candidate release against this plan's gate using public team fixtures and the pinned revisions. Exercise the browser, backend and ETL as a customer would; check authoritative state and absence of unauthorized effects. Keep real ML integration and stub mode distinct. Report each failure with reproduction and affected gate. Do not edit ML internals or waive acceptance requirements. Only the integration owner mutates the shared release stack.

Provider-neutral deployment and release

Recommended initial architecture: one approved Linux host running the existing separate ETL and backend images plus PostgreSQL, with a TLS gateway serving the built React application and proxying /api to the backend. The ML adapter reaches the team's separately delivered service or local runtime according to its contract. PostgreSQL stays on an internal network. Persist database/simulator state; inject runtime secrets from private files; use public team fixtures for the externally accessible demo.

This extends the existing Compose architecture and minimizes new operational seams. Docker documents [single-server Compose deployment and production overrides](#) and [runtime secret files](#). It is a hackathon deployment design, not evidence of production-bank readiness.

Before P14, provide an approved hosting account/host, DNS or a supported public TLS address, required access, a spending ceiling if any, and the ML service's connection information through private configuration. No account creation, purchase or public deployment was performed while preparing this plan. If prerequisites are absent, finish the reproducible bundle and video preparation but keep G4 blocked; a local video does not fulfill the deployed-link requirement.

P13 acceptance includes a clean checkout without organizer raw files, local outputs, private ML drafts or nested workspace assumptions; synthetic seed through the ETL; healthy API; static frontend/API routing; restart persistence; and recorded repository commit IDs, locks, image digests, data release, policy/contract versions and test evidence. Validate rollback of a candidate release and bound demo-state retention. Treat Compose secret bind mounts as private runtime files, not as an encrypted secret-management system.

For P15, assemble a public integration repository named factored-hackathon-2026-[team-name] that references pinned ETL/BCK/FRT sources and provides a single bootstrap entry point. Inspect public contents/history for restricted records and secrets. Do not rely on ignored nested repos being included in a parent clone. Include a navigation index for the three component repos, deployment, aggregate data-quality evidence, architecture, simulator limitations, engineering acceptance and the ML team's separately supplied comparison.

Suggested 4-6 slide story: data-backed problem and limitations; the three-route customer demonstration; architecture and authority boundaries; measured engineering/ML evidence with owners and provenance; operational tradeoffs and remaining deployment work. Record the video from the deployed accepted build, showing both languages, clarification and structured escalation. Explain permitted simulated actions and the distinction between a registered request and a completed banking operation.

The kickoff lists email submission, but this planning request does not authorize sending it. Prepare and verify the submission package; record transmission/receipt as a separate external step when authorized.

Requirements outside this sprint and the final release condition

The ML team must supply the compatible runtime, model/prompt versions where applicable, valid label/relevance provenance, leakage controls, baseline and proposed results on the same held-out workload, language-specific results, safety/failure results and latency/cost assumptions relevant to their component. If model judging is used, the original brief requires validation against human **or deterministic** judgments; agent review alone must not be mislabeled as human certification. Backend instrumentation can supply execution timing and verified tool events, but it cannot certify model quality.

The engineering sprint is accepted when G0-G3 pass and the final runtime connects successfully through the ML interface. The **hackathon submission** is accepted internally only when G4 also passes, the required external ML evidence is present, and all required links/artifacts match the frozen build. Do not claim the full hackathon solution is complete merely because engineering tests with a stub pass.

The first dispatch should be **P00**, with **P03** in parallel. Once the contract is frozen, dispatch **P01** synthetic boot; after it passes, dispatch **P02**, **P04** and **P10** using independent ownership. P05 then integrates the first complete browser-to-backend verified pause over an ETL-produced synthetic release.

GitHub branch and PR review - October 3, 2026

Reviewed through GitHub CLI authenticated as **yehosuah (Yehosua Hercules)**. Snapshot: October 4 at 04:18 UTC, October 3 at 22:18 America/Guatemala. The three established repositories are ETL (FactoredAI_base), backend (FactoredAI_BCK) and frontend (FactoredAI_FRT); the workspace's ETL is the repository interpreted from the request's "CTO" reference.

Decision: reuse the ETL candidate and the backend branch chain, with the fixes and integration gates below. The implementation is a useful foundation, but the browser-to-conversation-to-confirmed-action demonstration is still missing. No application code was changed or merged during this review. The only published change is this guide and its exports. ML implementation remains excluded.

Inventory and verdicts

There are **zero open PRs** in each repository. All nine remote branch heads were inventoried. The implementation PRs remembered in the request are already merged. Links and commit IDs below pin the reviewed code rather than promising that future branch contents are identical.

Repository / branch	Reviewed revision	Assessment
ETL main	dbb32f5	PR #2 merged October 2: strong publication/reconciliation foundation. Historical live-stack results remain historical.
ETL andrew/ml-routing-evaluation	de0d3ea	Three unmerged commits: aggregate dataset profiler, optional ML pack and auxiliary-file hardening. Despite its name, this diff is ETL/data-quality work; no model implementation. Recommend integration after normal PR/CI review and synthetic boot acceptance.
ETL yehosuah/hackathon-agent-sprint	94ebbe5 before this guide update	Documentation and portable planning artifacts only; no additional system implementation.
BCK main	d6aa84a	PR #1 merged October 2: authenticated cards, historical movements, simulated actions and persistence. Good foundation, incomplete conversational product.
BCK yehosuah/card-support-integration	e59182c	Already contained in main; the one-commit difference is main's merge commit. Do not schedule this as new implementation.
BCK andrew/backend-observability	bd2e74a	One unmerged commit: bounded HTTP timing/counts and committed-action aggregates. Useful engineering instrumentation with explicit limitations.
BCK andrew/backend-tools	8328d74	Contains observability plus controlled tool dispatch. Useful authenticated seam; fix contradictory receipt acceptance before consuming it as proof of execution.
BCK andrew/human-handoff-routing	fd40d2c	Contains both preceding branches plus persistent cases, agent sessions, deterministic routing and lifecycle. Useful implementation; request changes for recovery and queue behavior, then complete deployment/conversation integration.
FRT main	95a1a43	Only remote branch and no PR history. Buildable ES/PT starter; no chat, API client, login, confirmations, receipt or support experience.

ETL [PR #1](#) is also merged, on September 30. Its draft ML-pack review state was checked as an ETL consumer dependency; model quality was not reviewed. The reviewed code heads have no GitHub check runs;

ETL/backend candidate combined statuses contain zero status contexts. A returned pending state with zero contexts is **absence of a CI signal**, not an observed failing job.

Findings to revisit

R1 - P1: accepted handoffs can become stranded

[Agent authentication](#) denies disabled or newly ineligible accounts. That is correct access revocation. But [case transitions](#) prohibit customer cancellation after acceptance, and [administrative reroute](#) only permits queued cases.

Reproduced on disposable PostgreSQL: create -> assigned -> accepted; disable its assigned account; agent resolution returns 401, customer cancellation returns 409, administrative reroute returns 409. The case remains accepted with no supported recovery operation. Source refresh can cause the same eligibility failure; that refresh variant was inferred from the shared eligibility check, not separately exercised.

Required: an authorized, audited reassignment/requeue recovery path that preserves case history and cannot be driven by the model or customer. Test disabled accounts, release-driven ineligibility and concurrent lifecycle transitions. Owner: BCK, P08/P10.

R2 - P2: terminal cases occupy the active queue

[Handoff listing](#) sorts by priority and oldest creation without excluding terminal states. It is valid as case history, but insufficient as the agent work queue required by the sprint.

Reproduced: resolve a case, create a new case at the same priority, fetch /agent/handoffs?limit=1; the resolved case appears first and the assigned case moves to the next page. High-priority terminal history can similarly hide lower-priority open work.

Required: separate active queue from history using an explicit status filter or active-first ordering, with pagination tests. Unassigned cases also need a restricted operational queue view: current customer/agent transports cannot provide a global manual-review queue. Owner: BCK/FRT, P08.

R3 - P2: contradictory action state is labeled verified

[Dispatcher receipt validation](#) checks action, product, simulated status and outcome, but accepts any state from the broad state enum. This behavior is also present in backend-tools.

Reproduced at the Store seam with a controlled adapter: `pause_card` receives a receipt whose action is `pause` and whose state is `ACTIVE`; the result is `ok=true`, `verified=true`. The unit suite also uses a `PAUSED` receipt for several different state-changing tools. This is a validation weakness at the adapter seam, not evidence that the real Store currently commits incorrect state transitions.

Required: match state-changing action receipts to their expected states: `pause` -> `PAUSED`, `block` -> `BLOCKED`, `activate/reactivate` -> `ACTIVE`. Keep request-only semantics for replacement/charge review. Add contradictory-state and valid-replay checks. Owner: BCK, P04/P05/P09.

Integration gaps that tests do not close

- **Deployment grants and readiness:** handoff code reads `bank.customers.segment` and `bank.service_agents`. The existing ETL backend grants do not include these columns. The branch supplies a least-privilege inherited-role [SQL script](#), but it is a separate administrative step, absent from current integrated Compose boot. `/health/ready` still checks only the Store release; it does not invoke `check_configuration()` for handoff access. Wire preflight/provisioning into reproducible boot and prove failure when required grants are absent. Do not solve this with table-wide grants.
- **Conversations and confirmation:** tool dispatch is internal and synchronous; it is not a chat transport or an ML runtime. Mutating tools have no server-owned pending-confirmation contract. Build P04/P05 around them, authenticate outside model context, and offload synchronous calls from async orchestration.

- **Case scope:** persistent cases now capture summary, context and unresolved questions, with model claims separated from allowlisted committed evidence. They still take up to 20 customer/card actions, with no conversation ID or failed/unknown attempts. Reuse the case module and add conversation-scoped links; do not claim P08 is complete.
- **Demo data and provenance:** optional ML packaging removes one private-file prerequisite. It does not create the complete nine-table synthetic boot, fixture movements, isolated visitor state or source-aware backend provenance. The bank-products query still hardcodes organizer_synthetic for ETL-published products; fix this before serving team-generated ETL fixtures.
- **Measurements:** HTTP percentiles cover recent process-local samples up to response headers. Counters reset on restart; workers do not share them. Persistent aggregates count committed actions/cases, not attempted failures, model cost, containment or safe resolution. P11 still needs turn/tool outcome tracing, end-to-end timings and denominators.
- **Frontend and release:** all browser work, integrated clean-clone CI, public TLS deployment and submission artifacts remain outstanding. The three repositories are private; publication of the final safe submission repository is still a planned gate.

Validation performed

Revision	Evidence from this review
ETL de0d3ea	make check passed; make test: 187 passed , no duplicate test copies in the isolated checkout. Profiler CLI succeeded on the existing accepted Parquet release without changing data or publication pointers.
BCK observability bd2e74a	Lock/lint/format checked; 48 passed, 7 skipped on this tip. Explicit Ruff checks used --no-respect-gitignore because the archive lived under an ignored review-output directory.
BCK tools 8328d74	Lock/lint/format checked; 112 passed, 21 skipped on this tip, with the same explicit Ruff handling.
BCK handoff fd40d2c	make check: 172 passed, 43 skipped with default local-binary fixture. Then 215 passed, zero skipped with disposable PostgreSQL 17.11. Only infrastructure-fixture selection was adapted; original application and test assertions were unchanged and restored afterward. Two additional PostgreSQL probes reproduced R1/R2; a controlled-adaptor probe reproduced R3.
FRT 95a1a43	npm run check passed: lint, TypeScript build and Vite production build. Browser workflow acceptance remains pending because no workflow implementation exists.

Temporary databases used invented fixtures only, bound to localhost, with no customer datasets mounted. Containers were removed. These results are not a new public-deployment or integrated Compose acceptance pass.

The profiler independently confirmed 6,114,029 input = 6,020,768 accepted + 93,261 rejected rows. It found **1,547,408 ownership-consistent card transaction rows**, including **77,712 recorded declines (5.022%)**. This is historical transaction activity, not measured card-support demand or recoverable revenue. Accepted transcript data has **42 distinct customer texts across 143,852 rows** and **546 distinct full texts**; these aggregate data-quality observations must not be presented as independent conversation volume. No transcript text or individual identifier is included in this guide.

Revised agent dispatch

1. **P00 / integration:** pin ETL de0d3ea, BCK fd40d2c, FRT 95a1a43 as candidate revisions, with main revisions separately recorded. Review/fix R1-R3, reconcile handoff grants and readiness, and prepare application PRs through the team's normal workflow. Main stays the accepted baseline until integration gates pass.

2. **P03 / data:** reuse factored-profile; preserve the accepted-release/profile revision and aggregate report. Finish workflow rationale, product-mention attribution limitations and freshness evidence rather than rebuilding the profiler.
3. **P01 / data-runtime:** reuse optional ML packaging. Add the missing synthetic ETL release, movement/provenance support, restricted grants, agent account provisioning and isolated public-demo boot.
4. **P04 / service:** reuse ToolDispatcher and typed case interfaces; implement owned persistent conversations and the delivered ML adapter. **P05** adds authoritative confirmation and the verified browser pause.
5. **P08 / service-experience:** extend persistent cases with conversation/attempt references, R1 recovery, R2 active/manual queues and the support view. **P09/P10** validate R3, unknown outcomes and refresh/restart continuity.
6. **P11-P15 / acceptance-release:** reuse backend instrumentation while collecting missing system-level evidence, completing frontend polish, clean-clone CI, provider-neutral deployment and the required slide/video package.

The October 3-5 calendar, dependency gates, work-in-progress cap and exclusion of ML development remain in force. No packet is marked complete merely because a branch contains part of its implementation.